



Criptografía y Seguridad

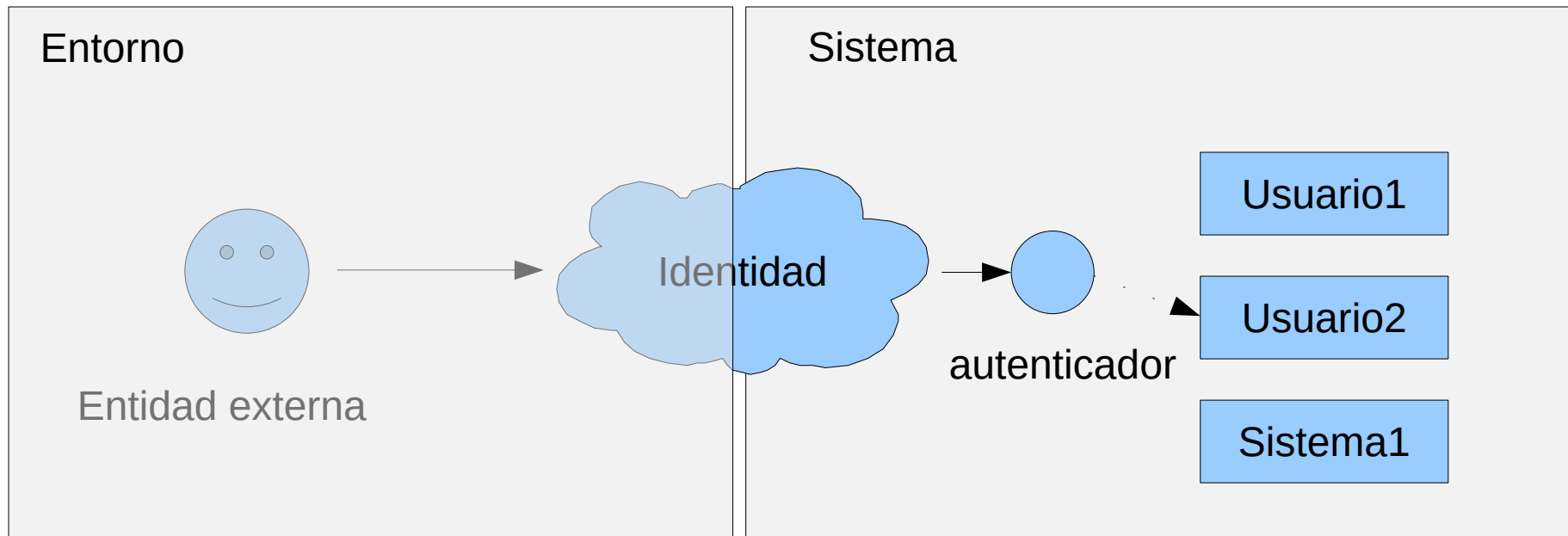
Seguridad en
aplicaciones:
Autenticación

Autenticación

Asociar una identidad a un principal

La identidad corresponde a una entidad externa

El principal es una representación interna del sistema



Confirmación de identidad

Pasos:

- Obtener información de identidad
- Analizarla
- Determinar si corresponden a la entidad

Consecuencias:

- Hay que almacenar información de cada entidad
- Hay que contar con mecanismos para procesar la información

Sistema de Autenticación

Componentes

$A = \{ a \}$ - Información de autenticación

La designación de la entidad e información provista

$C = \{ c \}$ - Información complementaria

Contiene la especificación de un principal e información almacenada por el sistema

$F = \{ F: A \rightarrow C \}$ - Funciones de complementación

Deriva información complementaria

$L = \{ L: A \times C \rightarrow \{ 0, 1 \} \}$ - Funciones de autenticación

Determina si un par A, C son una asociación válida

$S = \{ s \}$ - funciones de selección

Funciones que permiten crear y actualizar A y C

Información de Autenticación

Provista por una entidad externa

Posee diferentes grados de confianza

Fuentes o factores

Indican la naturaleza de la información

Factor: Algo que conozco

Ejemplos

- Clave

- Frase (passphrase)

- Pregunta secreta

Basan la identificación en secretos compartidos

- Requiere almacenar de manera segura el secreto

- Depende de la confidencialidad del secreto

- Uno de los mecanismos más empleados

- Sirve de base a otros mecanismos

Factor: Algo que tengo

Ejemplos

- Smart card/USB token

- Generador de claves (por ejemplo RSA-ID)

- Celular

- Tarjeta de coordenadas (Token Grid)

Basan la identificación en algún elemento físico

- Requiere el cuidado del elemento

- El elemento puede ser copiado / clonado

- La información en transito puede ser suplantada

Factor: Algo que soy

Ejemplos

Huellas digitales

Retina

Voz

Cara

Basan la identificación en características intrínsecas de la entidad

Las características no son fácilmente modificables

Los métodos actuales no son 100% eficaces

Asumen que el dispositivo de lectura no puede ser manipulado

Factor: Contexto

Ejemplos

Red de origen/ Pais y ciudad / Geolocalización

Host / Terminal / Device de origen

Dia y hora de acceso, Velocity

Basan la identificación en el contexto de la información que se recibe

Pueden actuar como factores positivos

Un operador debe conectarse desde la red privada del centro de computos

O como factores negativos

Si un usuario accede desde otro pais hay MENOS chances de que sea quien dice ser

Ejemplo de sistema de autenticación

Autenticación con claves (ejemplo ilustrativo)

$A = \{ x / x \text{ es clave} \}$

$C = h(A)$

$F = \{ h(x) \}$

$L = \{ == \} \quad \leftarrow \text{Esto significa } F(A)=C \text{ es valido}$

$S = \{ \text{adduser()}, \text{removeuser()}, \text{passwd()} \}$

Claves - Tipos

Secuencias de caracteres

Con restricciones. Ej: 8 caracteres, 10 dígitos, etc

Generadas aleatoriamente, por el usuario o de manera asistida

Secuencias de palabras

Llamadas frases clave (pass-phrases)

Algorítmicas

Pregunta-respuesta (challenge-response)

Claves de uso único (OTP – One time passwords)

Claves - Almacenamiento

Como texto en claro

Puede ser en un archivo o en una base de datos

Puede ser accedido por fuera del sistema

No puede garantizarse confidencialidad de las claves

NO RECOMENDADO

Almacenamiento

Archivo cifrado

Requiere una clave para acceder a las contraseñas

La clave puede estar

- En un archivo de configuración

- En el mismo ejecutable del sistema

- Ser ingresada cada vez que inicia el sistema

- En un dispositivo criptográfico especializado

Solo recomendado si es requisito acceder a la clave original

Por ejemplo, para reenviarla a un tercer sistema

Mas adelante veremos la forma correcta de
(no) almacenar contraseñas: PBKDF2

Ejemplo: Sistema unix (legacy)

Se almacena por cada usuario el resultado de una de 4096 funciones de hash

$A = \{ \text{secuencias de hasta 8 caracteres} \}$

$C = \{ \text{xxHHHHHHHHHHHH} \}$

xx = funcion de hash (0-4095) – 2 caracteres

HH...H = Resultado de la función – 11 caracteres

- $F = \{ \text{DES}_{\text{xx}}(.) \}$
- $L = \{ \text{login, su, sudo, ...} \}$
- $S = \{ \text{passwd, adduser, ...} \}$

Ataques a un Sistema de autenticación

Objetivo del sistema: identificar correctamente a entidades

Objetivo de un atacante: ser identificado como una entidad (impersonarla)

Mecanismo: Encontrar $a \in A$ tal que:

Para algún $f \in F$, $f(a) = c$

c está asociado a una entidad

Verificación:

Probar varios a , computar $f(a)$ y comparar con c

Intentar autenticar via $I(a) \in L$

Ataques
offline

Ataques
online

Ataques: Adivinando claves

Prueba y error de una lista de claves potenciales

Offline: se conoce f y c y se prueban para diferentes a si $f(a) = c$

Ejemplo, obteniendo el archivo `/etc/shadow` de Linux, con programas como crack y john the ripper

Ejemplo, obteniendo el archivo SAM de windows, con programas como ophcrack

Online: Haciendo uso de la función L , probando con diferentes a , hasta pasar la autenticación

Ejemplo, probado la función de login con una cuenta conocida (root, administrator, guest)

Prevenciones (general)

Esconder información para que a, c o f no sean conocidas simultáneamente

Por lo general f es conocida o puede conocerse

Es más fácil proteger c que a, que está en manos de la entidad externa

Ejemplo: /etc/shadow en unix

Prevenir el uso a la función de autenticación

Tiempos crecientes ante fallas en la autenticación

Deshabilitar principales

Jailing / Honeypot

Captchas

Prevenciones (general)

Subir la complejidad de las claves

Formula de Anderson: $P = TG / N$

P = probabilidad de adivinar una clave

G = número de pruebas realizables por segundo

T = tiempo dedicado a las pruebas

N = tamaño del espacio de claves

Se puede utilizar para estimar el tiempo de un ataque

Ejercicio

Considerar:

Claves numéricas de 10 dígitos

Se pueden probar 10.000 claves por segundo

¿Cual es el tiempo de búsqueda estimado para tener una probabilidad > 0.5 ?

$$P \geq TG / N \Rightarrow T \leq PN / G = 0.5 * 10^{10} / 10.000$$

$$T \leq 500.000s \sim 6 \text{ dias}$$

Ejercicio (2)

Considerar:

Claves de 8 letras (26 posibles caracteres)

Se pueden probar 10.000 claves por segundo

¿Cual es el tiempo de búsqueda estimado para tener una probabilidad > 0.5 ?

$$P \geq TG / N \Rightarrow T \leq PN / G = 0.5 * 26^8 / 10.000$$

$$T \leq 1.044.135s \sim 121 \text{ días}$$

Ejercicio (3)

Considerar:

Claves de alfanuméricas (36 posibles caracteres)

Se pueden probar 100.000 claves por segundo

¿Qué longitud mínima se debe requerir para que la probabilidad de que un atacante encuentra la clave en menos de 1 año sea menor a 0.5?

$$P \geq TG / N \Rightarrow$$

- $N \geq TG / P = (365 * 24 * 60 * 60) * 100.000 / 0.5$
- $N = 36^L \geq 6,3 * 10^{12}$
- $L \geq \log_{36} (6,3 * 10^{12}) \sim 8.22 \Rightarrow L \geq 9$

Mejores ataques

Hasta ahora se asume una búsqueda aleatoria

Mejoras:

- Utilizar diccionarios de palabras

- Utilizar diccionarios de claves descubiertas

- Utilizar transformaciones simples de palabras

 - Sufijos

 - Prefijos

 - Reemplazar l -> 1, o -> 0, vocales por números

 - Palabras espejadas

 - Dos palabras combinadas

- Utilizar información de contexto

 - Nombre, usuario, dni, fecha de nacimiento, etc

Selección de claves - alternativas

Selección aleatoria

Condición ideal – cada clave es equiprobable

Difícil de memorizar (ej: fL3K&8%j”)

Claves pronunciables

Palabras sin sentido, pero formadas por fonemas

Ejemplos: helgoret, mipoterjo, jusacila

Fáciles de memorizar

Problemas: el espacio de claves se ve muy reducido

Permitir que el usuario elija

Problema: las claves tienden a ser fáciles de adivinar

Revisión proactiva de claves

Consiste en el análisis de una clave al momento de crearla

- Permite detectar y rechazar claves 'fáciles'

- Debería permitir aplicar reglas sobre palabras

- Debería permitir realizar búsquedas en diccionarios

- Debería permitir realizar búsquedas de patrones

- Debería utilizar información sobre el usuario y el contexto

Expiración de claves

Forzar a los usuarios a cambiar de clave luego de algún tiempo o evento

Limita el daño producido por una clave comprometida

Requisitos

Evitar el reuso de claves (recordar y bloquear las N últimas)

Evitar que se pueda cambiar de clave varias veces en un periodo de tiempo

Dar a los usuarios tiempo para pensar una nueva clave

- No forzarlos a cambiar clave al momento de login
- Avisar con anticipación

Salting

Escenario:

Un atacante consigue $c_1, c_2, \dots, c_n$

Cada prueba $a_i \rightarrow f(a_i)$ puede probarse en paralelo contra todos los c_j

Objetivo: aumentar el tiempo requerido para adivinar claves

Método: introducir una perturbación

$$f(a) = x \parallel f'(a, x)$$

Cada c se calcula con una perturbación diferente

El atacante no puede reutilizar $f(a)$ para buscar varias cuentas en paralelo

PBKDF2 (PKCS 5 v2.0)

Idea

Sea *pass* una contraseña

Calcular un Salt: S

Utilizar una función pseudoaleatoria
(criptosistema simétrico o MAC):

$$u_1 = \text{PRF}(\text{pass}, S)$$

Aplicar la función iterativamente:

$$u_2 = \text{PRF}(\text{pass}, u_1)$$

...

$$u_j = \text{PRF}(\text{pass}, u_{j-1})$$

Resultado: $u_1 \oplus u_2 \oplus \dots \oplus u_j$

PBKDF2 (PKCS 5 v2.0)

Algoritmo:

$$K = \text{PBKDF2}(\text{prf}, \text{pass}, \text{salt}, c, \text{len})$$
$$K = T_1 \parallel T_2 \parallel \dots \parallel T_n \quad (n = \text{len} / |\text{prf}|)$$
$$T_i = u_1 \oplus u_2 \oplus \dots \oplus u_c$$
$$u_1 = \text{PRF}(\text{Pass}, \text{Salt} \parallel \text{INT32_BE}(i))$$
$$u_k = \text{PRF}(\text{Pass}, u_{k-1})$$

TIP: En java, JCE implementa PBKDF2:

```
PBEKeySpec spec = new PBEKeySpec(password, salt, iterations, bytes * 8);  
SecretKeyFactory skf = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1");  
byte[] key = skf.generateSecret(spec).getEncoded();
```

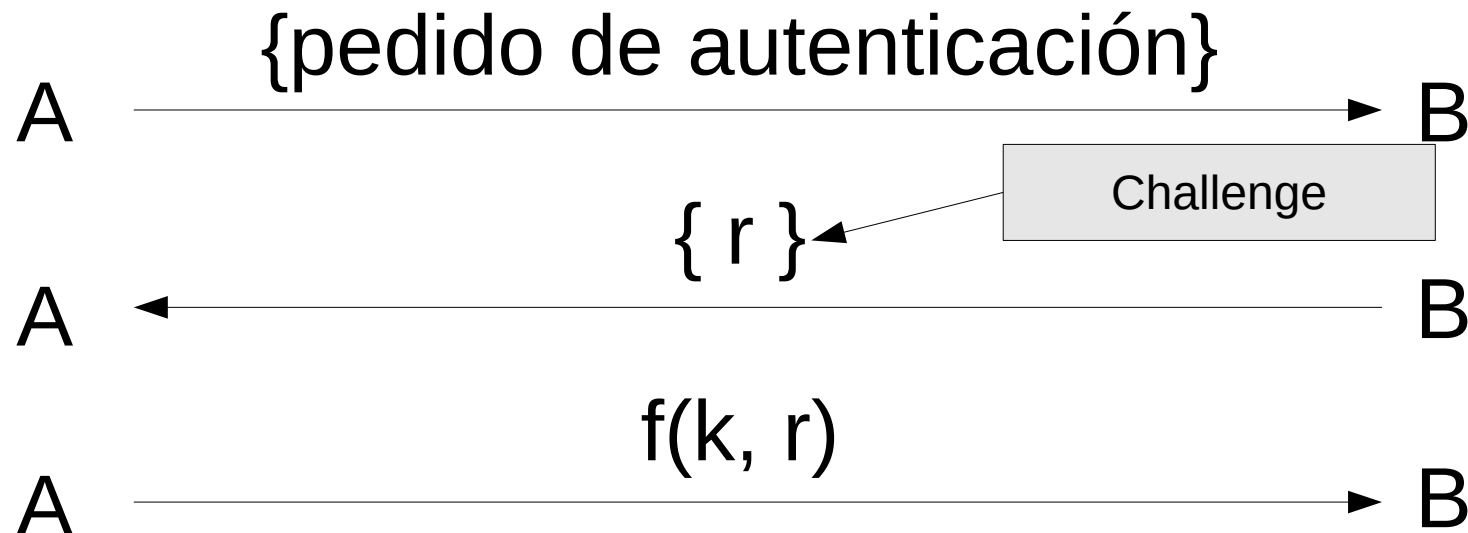
Challenge - Response

Problema: autenticar a un usuario remoto requiere que se envíen las claves

Requiere un canal seguro

Descubrir una comunicación antigua puede comprometer la clave

Solución: no enviar la clave

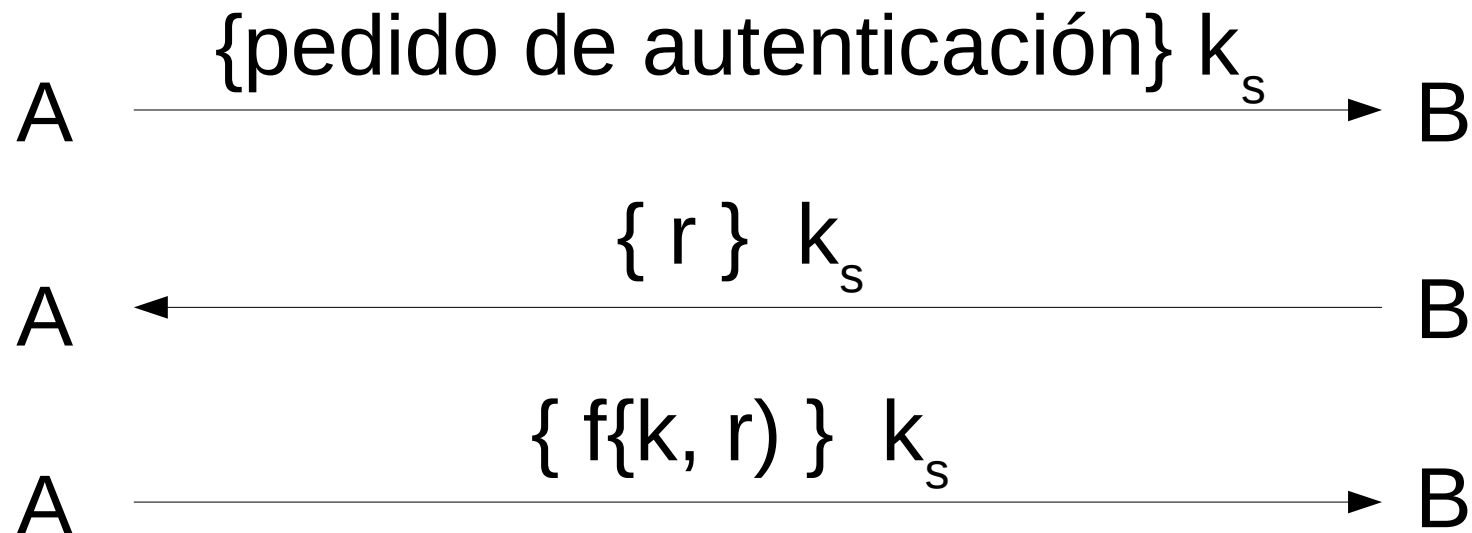


EKE – Encrypted Key Exchange

Objetivo: impedir que un atacante pueda adivinar claves capturadas en la red

Mecanismo: realizar el dialogo en un canal encriptado

El atacante no tiene forma de saber si una clave es correcta



OTP – Claves de uso único

Objetivo: Generar claves que sirven una única vez

¡No confundir con
One Time Pad!

A y B comparten una clave K y un contador C

Concepto:

$$(K, C) \rightarrow \text{OTP} \rightarrow (\text{otp}, C')$$

Dos variantes estandarizadas:

HOTP $\rightarrow$ hmac based OTP

TOTP $\rightarrow$ time based OTP

HOTP – RFC 4226

OTP basado en hmac

$\text{HOTP}(K, C) = \text{Truncate}(\text{HMAC-SHA1}(K, C))$

Truncate: extrae 32 bits de los 160 del mac

$\text{Offset} = \text{mac}[19] \& 0xf \leftarrow 4 \text{ bits del ultimo byte}$

$\text{bin} = \text{mac}[\text{offset} \dots \text{offset}+3] \& 0x7fffffff$

$\text{Result} = \text{bin} \% 10^n \quad (n = \text{cantidad de dígitos})$

HOTP – RFC 4226

OTP basado en hmac

$\text{HOTP}(K, C) = \text{Truncate}(\text{HMAC-SHA1}(K, C))$

El contador se incrementa en cada uso

Requiere sincronización (look-ahead)

Se aconseja usar throttling

Genera códigos de 1 a 9 dígitos

Se recomienda usar al menos 6

TOTP – RFC 6238

OTP basado en timestamps

$$\text{TOTP}(K) = \text{HOTP}(K, T)$$

$$\text{HOTP}(K, C) = \text{Truncate}(\text{HMAC-SHA-1}(K, C))$$

$$T = (\textit{current unix time} - T_0) / X$$

T_0 : tiempo de referencia

Usualmente, $T_0=0$

X : cantidad de segundos de vida de cada código

Usualmente $X=30$

TOTP – RFC 6238

OTP basado en timestamps

$$\text{TOTP}(K) = \text{HOTP}(K, T)$$

No requiere un contador

Se aconseja utilizar un drift-window

Cantidad de ticks desfasados aceptados

Se aconseja usar throttling

Genera códigos de 1 a 9 dígitos

Se recomienda usar al menos 6

Multi factor authentication

Consiste en requerir dos factores de diferente tipo

Racional: Cada tipo de factor requiere comprometer distintos assets

Combinaciones comunes:

- Algo que conozco + algo que tengo

- Algo que conozco + algo que soy

- Algo que tengo + algo que soy

Autenticación remota (SSO)

Consiste en la delegación de la autenticación en un sistema externo

Implica una relación de confianza

Productos:

- Active Directory Federation Services

- CAS

- Siteminder

Cada uno utiliza una tecnología diferente

Tecnologías abiertas

- OpenID 1 y 2 → Obsoletos

- OpenID Connect → Basado en OAuth2

Lectura Recomendada

Capítulo 12-13
Computer Security Art and Science

Matt Bishop